

Lab 8 — Drift and Observability Monitoring

Track B (BMD-45 vehicle detection) · Week 8 · DS5619 Machine Learning Systems Operations

Setup

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python generate_for_student.py --student-id <your roll number or institute email>
```

This overwrites `data/fixtures/` with your own synthetic CCTV images — same two camera profiles, seeded deterministically from your student ID. The script sanity-checks its own output with the mock detector (not your `drift_monitor.py`) and retries with a new sub-seed if the resulting camera_A vs camera_B drift would come out too small to be meaningful, so the lab's drift signal survives reseeding no matter which student ID you use.

Record your `--student-id` value in `NOTES.md`.

Files

- `src/drift_monitor.py` — implement the four `# TODO` functions.
- `src/mock_detector.py` — dependency-light stand-in detector (provided, don't edit).
- `src/run_pipeline.py` — complete driver script. Don't edit.
- `data/fixtures/camera_A_daylight/`, `data/fixtures/camera_B_lowlight/` — your two personalized synthetic camera conditions, generated above (don't hand-edit).

Background

BMD-45's paper reports a real domain-shift finding: a detector trained on a different dataset scores only ~33.6% mAP when evaluated on BMD-45's real-world CCTV footage, versus ~83.8% mAP for a detector trained in-domain on BMD-45 itself — the same kind of train/deploy mismatch that drift monitoring exists to catch. `data/fixtures/camera_A_daylight/` and `data/fixtures/camera_B_lowlight/` are a synthetic, illustrative stand-in for that idea — same `mock_detector.py`, two different visual conditions, deliberately built so the confidence-score distribution differs between them.

You'll treat `camera_A_daylight` as the reference distribution (what you'd expect from training-time-like conditions) and `camera_B_lowlight` as live traffic you're monitoring.

Your task

`src/drift_monitor.py` (four functions marked `# TODO`, each has a full docstring spec, ~50 min)

- `extract_confidence_scores(camera_dir)` — run the detector over a camera's images, collect the confidence score of every detection.

- `compute_psi(reference_scores, live_scores, n_bins)` — the PSI formula itself: bin both distributions, compare bin proportions.
- `classify_drift(psi)` — turn a PSI number into `"none"` / `"moderate"` / `"significant"` using the standard thresholds (given as constants).
- `summarize_scores(scores)` — count/mean/std/min/max for the report.

```
python src/run_pipeline.py
```

Writes `drift_report.json` comparing the two cameras (`src/run_pipeline.py` is complete, don't edit it).

Self-check

```
pytest tests/ -q
```

This is a self-check, not the grader.

Deliverables (what you commit)

- `src/drift_monitor.py`, completed.
- `drift_report.json` (from running the pipeline).
- A short `NOTES.md`: the `--student-id` value you used (required — see above), what drift level did the report show, does that match what you'd expect given the two cameras were built with deliberately different visual statistics, and what would you monitor IN ADDITION to confidence score if you had access to ground-truth labels a day later (tie this to the kinds of drift from the lecture — which one does confidence-score-only monitoring miss)?

Grading checklist

- ☐ `data/fixtures/` matches what `generate_for_student.py --student-id <NOTES.md value>` actually produces.
- ☐ `extract_confidence_scores` correctly wires the detector and is deterministic (checked against a held-out camera directory, not just the two provided).
- ☐ `compute_psi` returns ~ 0 for identical distributions and a large value for a fully shifted one (both checked independently).
- ☐ `classify_drift` respects both threshold boundaries exactly.
- ☐ `summarize_scores` handles the single-value edge case (`std=0`) without raising.
- ☐ `NOTES.md` correctly identifies which kind of drift confidence-score monitoring can and can't catch.
- ☐ Meaningful commit history and a working README.

Submission

```
git add -A
git commit -m "Week 8: drift and observability monitoring"
git tag week08-submit
git push origin main --tags
```

Source: this lab operationalizes the ML Observability and Drift Detection (PSI, univariate feature drift) content from the Week 8 lecture deck. Dataset: Sharma et al., "BMD-45: A Large-Scale CCTV Vehicle Detection Dataset for Urban Traffic," CVPR 2026 Findings.